Sprint 8 - Advanced Drupal Development Report

Prepared by: Aicha Nait Dayaa

Supervised by: Mr. Abdelmajid Bendrif
Mr. Hamza Bahlaouane

Date: March 16, 2026

Contents

1 Working with data (Entity API, Field API & Configuration API) 2
1.1 Entity API 2
1.2 Field API 6
1.3 Questions 9
2 Working with hooks 14
2.1 Introduction 14
2.2 Questions 14
3 Working with configuration & features 18
3.1 Introduction 18
3.2 Configuration Export and Import 18
3.3 Managing Environment Differences with Config Split 18
3.4 Features Module 21
3.5 Conclusion 24
4 Working with Cache 24
4.1 Introduction 24
4.2 Implementation 24
4.3 Conclusion 28
5 Working with Data Migration 28
5.1 Questions 28

1 Working with data (Entity API, Field API & Configuration API)

1.1. Entity API

Introduction

In this section, we create a custom content entity type **Contact** using Drupal's Entity API, covering the full structure of an entity module: definition, access control, forms, list builder, routing, and permissions.

Source Code

Custom Content Type - Source Code

Module Structure Overview

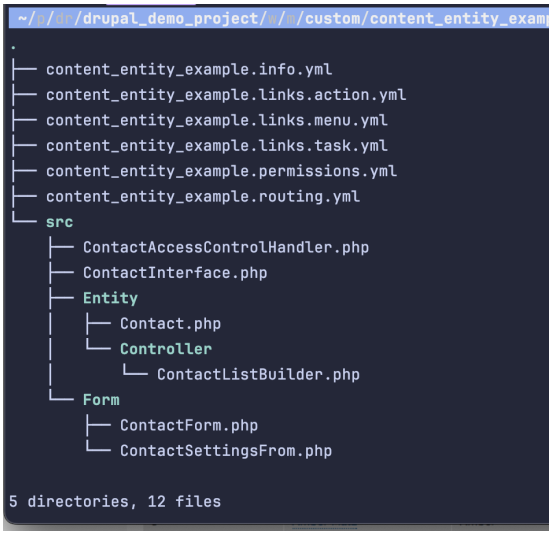


Figure 1: Module Structure

Entity Definition

The **Contact** entity is defined in `Entity/Contact.php` as a class extending **ContentEntityBase**, decorated with the **#[ContentEntityType]** attribute that declares its id, label, handlers, link templates, base table, and admin permission.

```
#[ContentEntityType(
  id: 'content_entity_example_contact',
  label: new TranslatableMarkup('Contact entity'),
  entity_keys: [
    'id' => 'id',
    'label' => 'name',
    'uuid' => 'uuid',
    'owner' => 'user_id',
```

```
  ],
  handlers: [
    'list_builder' => 'Drupal\content_entity_example\Entity\Controller\
      ContactListBuilder',
    'views_data' => 'Drupal\views\EntityViewsData',
    'access' => 'Drupal\content_entity_example\ContactAccessControlHandler
      ',
    'form' => [
      'add' => 'Drupal\content_entity_example\Form\ContactForm',
      'edit' => 'Drupal\content_entity_example\Form\ContactForm',
      'delete' => 'Drupal\Core\Entity\ContentEntityDelete',
    ],
    'route_provider' => [
      'html' => 'Drupal\Core\Entity\Routing\AdminHtmlRouteProvider',
    ],
  ],
  links: [
    'canonical' => '/content_entity_example_contact/{
      content_entity_example_contact}',
    'add-form' => '/content_entity_example_contact/add',
    'edit-form' => '/content_entity_example_contact/{
      content_entity_example_contact}/edit',
    'delete-form' => '/contact/{content_entity_example_contact}/delete',
    'collection' => '/content_entity_example_contact/list',
  ],
  admin_permission: 'administer contact entity',
  base_table: 'contact',
  field_ui_base_route: 'content_entity_example.contact_settings',
  list_cache_contexts: ['user'],
)]
```

Base fields are declared in **baseFieldDefinitions()**, defining the entity's database columns:

```
$fields['name']        // string field
$fields['first_name'] // string field
$fields['user_id']     // entity_reference to user
$fields['role']        // list_string (administrator / user)
$fields['created']     // created timestamp
$fields['changed']     // changed timestamp
```

The class also uses **EntityOwnerTrait** and **EntityChangedTrait**, and overrides **preSave()** to assign anonymous ownership when no owner is set.

Access Control

ContactAccessControlHandler extends **EntityAccessControlHandler** and overrides two methods:

- **checkAccess()** — maps `view`, `update`, and `delete` operations to their respective permissions, granting full access to users with the admin permission.
- **checkCreateAccess()** — checks the `add contact entity` permission for entity creation.

```
case 'view':
  return AccessResult::allowedIfHasPermission($account, 'view contact
      entity');
case 'update':
  return AccessResult::allowedIfHasPermission($account, 'edit contact
      entity');
case 'delete':
  return AccessResult::allowedIfHasPermission($account, 'delete contact
      entity');
```

Form

ContactForm extends **ContentEntityForm** and handles both `add` and `edit` operations. It adds a language selector to the form and overrides **save()** to redirect to the collection page and display a status message depending on whether the entity was created or updated.

```
if ($result == SAVED_NEW) {
  $this->messenger()->addStatus('New contact %label has been created.');
} else {
  $this->messenger()->addStatus('The contact %label has been updated.');
}
```

A separate **ContactSettingsForm** extending **FormBase** serves as the entity's admin settings page, referenced by the `field_ui_base_route` in the entity annotation.

List Builder

ContactListBuilder extends **EntityListBuilder** and defines the admin listing table. **buildHeader()** declares four columns — `id`, `name`, `first_name`, and `role` — while **buildRow()** maps each entity's field values to the corresponding columns. The **render()** method is overridden to prepend a description with a link to the settings page above the table.

Routing and Links

Routes are partly auto-generated by **AdminHtmlRouteProvider** via the `links` key in the entity annotation, and partly declared manually in `routing.yml`:

```
# Auto-generated via annotation links
/content_entity_example_contact/{id}          # canonical
/content_entity_example_contact/add           # add-form
/content_entity_example_contact/{id}/edit     # edit-form
/contact/{id}/delete                          # delete-form
/content_entity_example_contact/list          # collection

# Manually defined
/admin/structure/content_entity_example_contact_settings  # settings
```

Permissions

Permissions are declared in `.permissions.yml` and checked by the access control handler:

```
'administer contact entity' # full admin access
'view contact entity'       # view access
'add contact entity'        # create access
'edit contact entity'       # update access
'delete contact entity'     # delete access
```

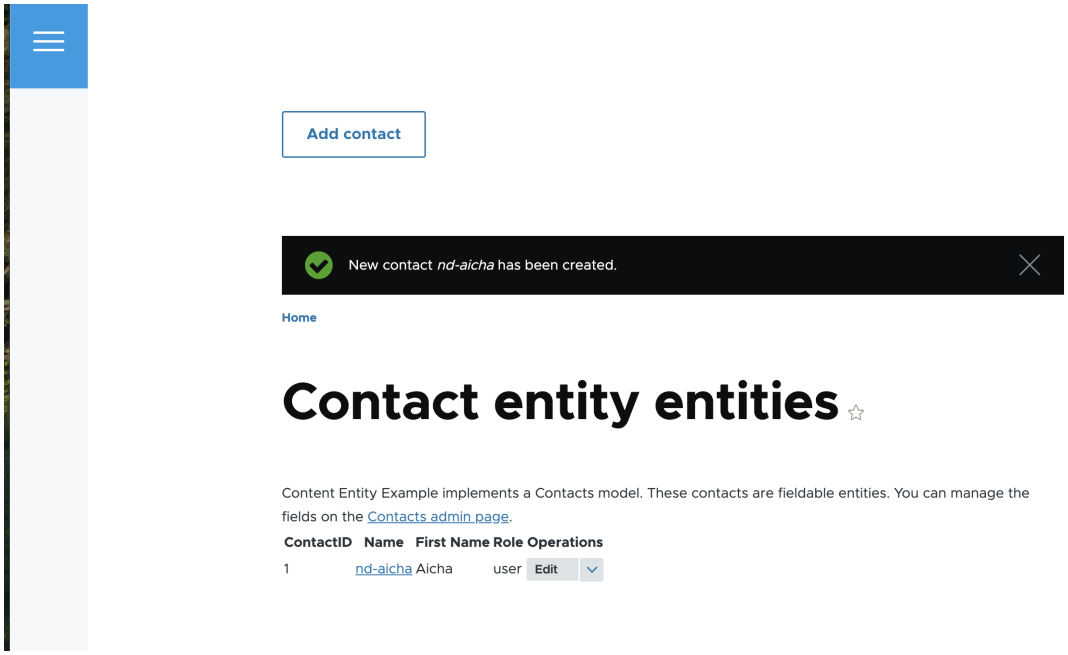


Figure 2: Contact Content Entity

1.2. Field API

Introduction

In this section, we create a custom field type using the Field API, covering its three main components: **Field Type** — defines the data structure and storage, **Field Widget** — defines the edit form presentation, and **Field Formatter** — defines how the data is rendered for the end user.

Source Code

Custom Field Type - Source Code

Module Structure

We implemented the custom field type inside a custom module, where each component has its own dedicated directory under **src/Plugin/Field/**, as shown in Figure 3.

Figure 3: module structure

Implementation

To create a background color field that accepts one of the two formats (HEX code, RGB code), we followed these steps :

1. **Creating the Field Type :**
   The field type defines a single column **value** of type **text**, which stores the color value as a string regardless of the format (HEX or RGB). The conversion of the format is handled at the widget level.

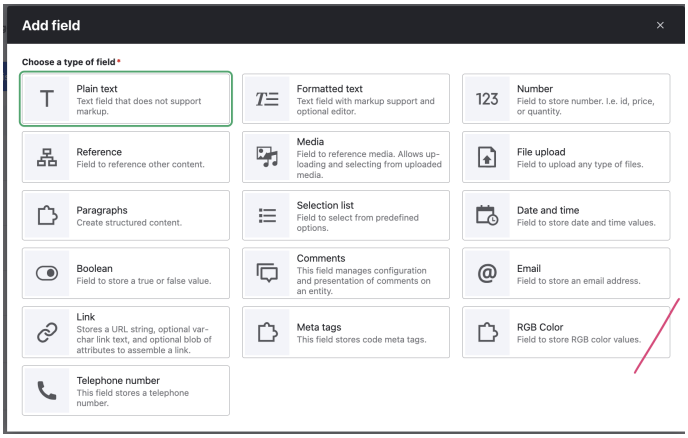


Figure 4: Creating the background color Field on the Article content type

2. **Creating the Field Widget :**
   We create two widgets, one for displaying the HEX code and the other for the RGB code.

   - A **HEX widget** : accepts a hex color code (e.g. `#ff0000`)
   - An **RGB widget** : accepts three separate numeric inputs for Red, Green, and Blue values

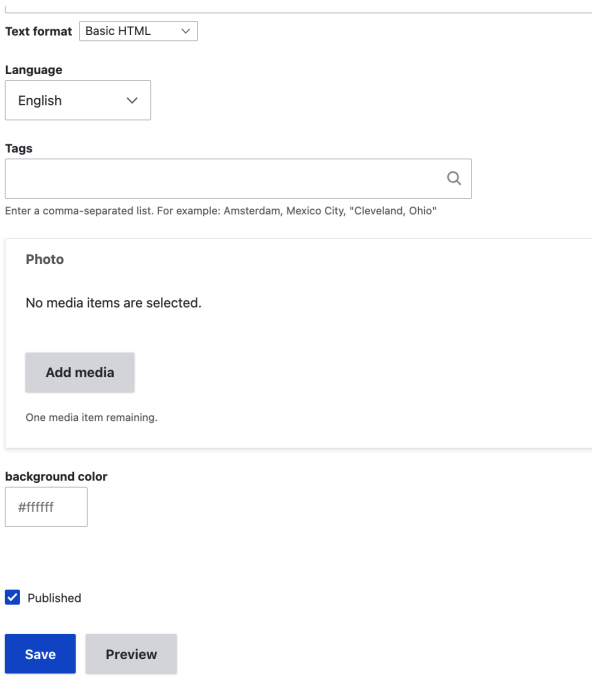


Figure 5: Field Widget for the HEX code on the edit form

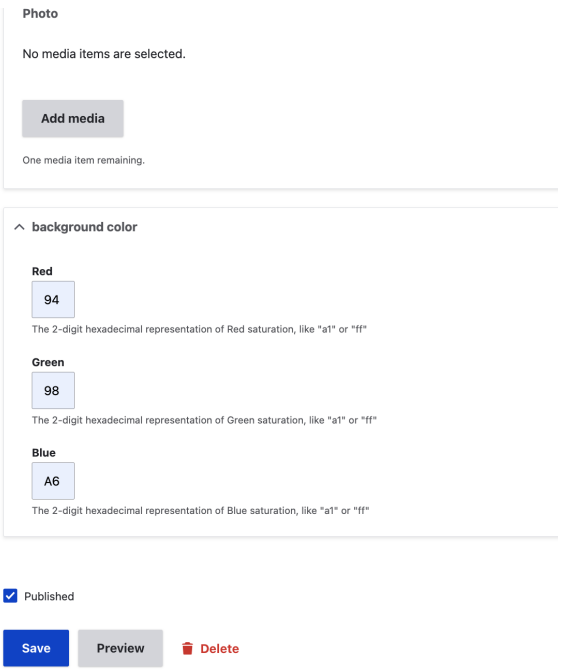


Figure 6: Field Widget for the RGB code on the edit form

Figure 7: Background color field widgets

3. **Creating the Field Formatter :**
   The formatter iterates over all field values and for each one renders an HTML **¡p¿** tag with the stored hex color applied as a `background-color` CSS style.

Figure 8: Background color field formatter

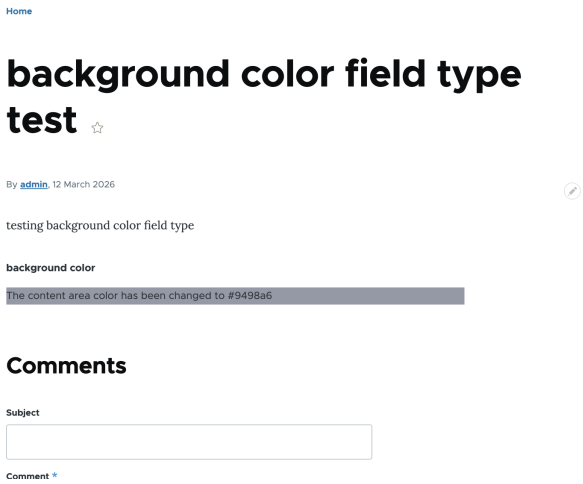


Figure 9: Background Color Field

1.3. Questions

Q1. Implementing the hook hook_entity_base_field_info_alter()

Pasword Policy Enforcer - Source Code

To add a constraint on the pass field of the user entity, without altering the entity's core code, we use the hook **hook_entity_base_field_info_alter**. This hook allows modules to alter base field definitions provided by existing entity types.

1. **Password policy Definition:**
   We first need to add the password policy and define its different constraints. These constraints will then be evaluated by the **password_policy.validator** service :

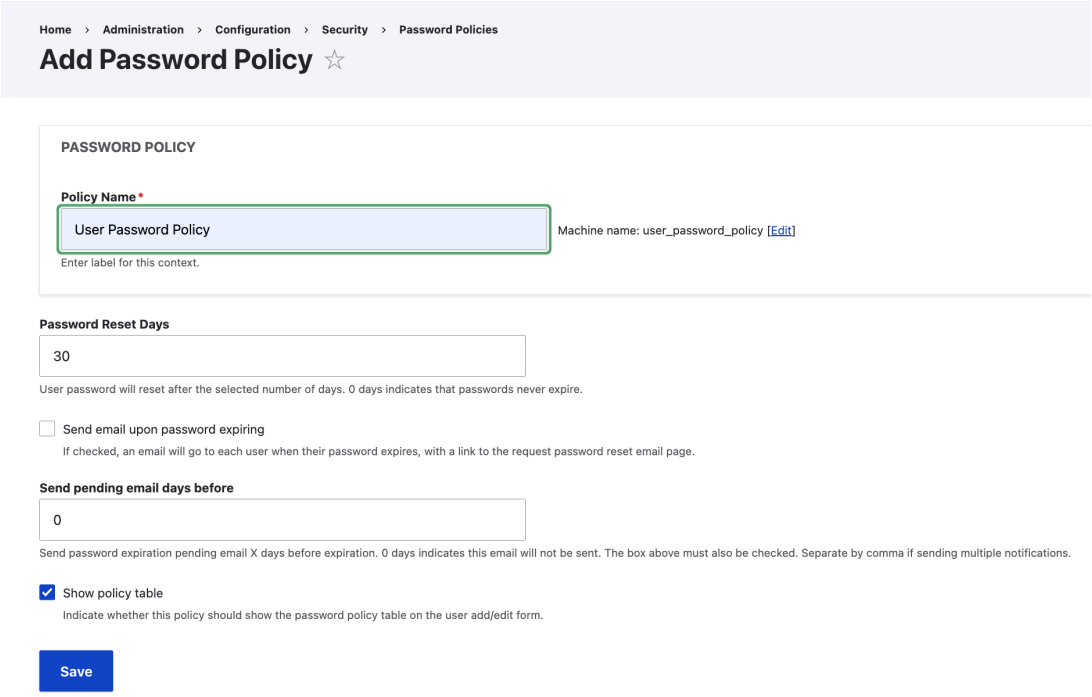


Figure 10: Creating a new password policy

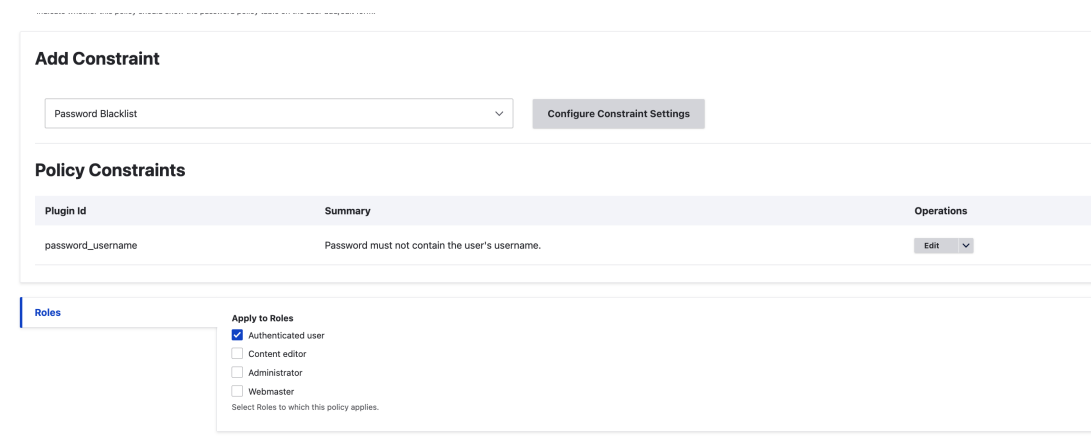
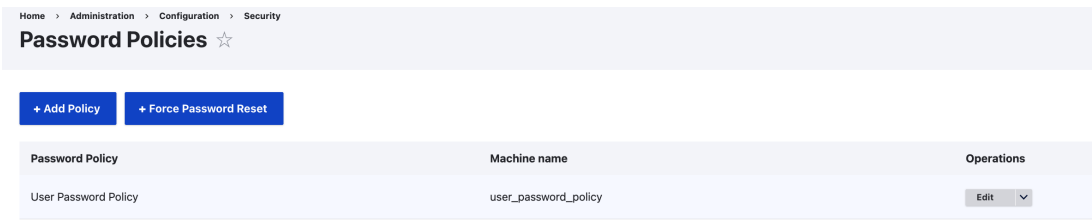


Figure 11: Adding the **password username** constraint to the policy.



2. **Implementing the hook hook_entity_base_field_info_alter() :**
   In the **.module** file, we implement the hook **hook_entity_base_field_info_alter()** that attaches the constraint to the password field of the user entity.

```php
/**
 * implements hook_entity_base_field_info_alter().
 */
function password_policy_enforcer_entity_base_field_info_alter(&$fields, $entity_type) : void {
  if ($entity_type->id() === 'user') {
    $fields['pass']->addConstraint('UserPasswordPolicyConstraint');
  }
}
```

3. **Defining the constraint in PasswordPolicyConstraint :**
   This class defines the constraint's metadata, including the plugin ID **UserPasswordPolicyConstraint**, it's label, the violation message shown when the password fails validation, and lastly the **ValidatedBy()** method that binds the constraint to its validator class **PasswordPolicyConstraintValidator**.

```php
#[ConstraintAttribute(
  id: 'UserPasswordPolicyConstraint',
  label: new TranslatableMarkup('Password Policy Constraint')
)]
class PasswordPolicyConstraint extends Constraint {
  public $message = 'The password does not satisfy the configured password policy.';

  /**
   * {@inheritdoc}
   */
  public function validatedBy(): string {
    return PasswordPolicyConstraintValidator::class;
  }
}
```

4. **Defining The constraint validator PasswordPolicyConstraintValidator :** This class is responsible for validating the password field. It injects the **password_policy.validator** service and uses it to check user's submitted password against all registered password policies.

   If the password violates any configured rule, a violation is raised.

```php
class PasswordPolicyConstraintValidator extends ConstraintValidator implements ContainerInjectionInterface {

  protected mixed $passwordPolicyValidator;
  public function __construct(mixed $passwordPolicyValidator)
  {
```

```
        $this->passwordPolicyValidator = $passwordPolicyValidator;
    }

    public static function create(ContainerInterface $container): static
    {
        return new static(
            $container->get('password_policy.validator')
        );
    }

    /**
     * {@}}
     */
    public function validate(mixed $value, Constraint $constraint): void
    {
        if (!$constraint instanceof PasswordPolicyConstraint) {
            return;
        }

        $password = $value->value ?? NULL;
        if (empty($password)) { return; }

        $user = $value->getEntity();
        if (!$user instanceof UserInterface) { return; }

        $violations = $this->passwordPolicyValidator->validatePassword($password, $user);

        if (!empty($violations)) {
            $this->context->addViolation($constraint->message);
        }}}
```

To verify that the validation workflow functions correctly, we attempt to change a user's password so that it contains the username :

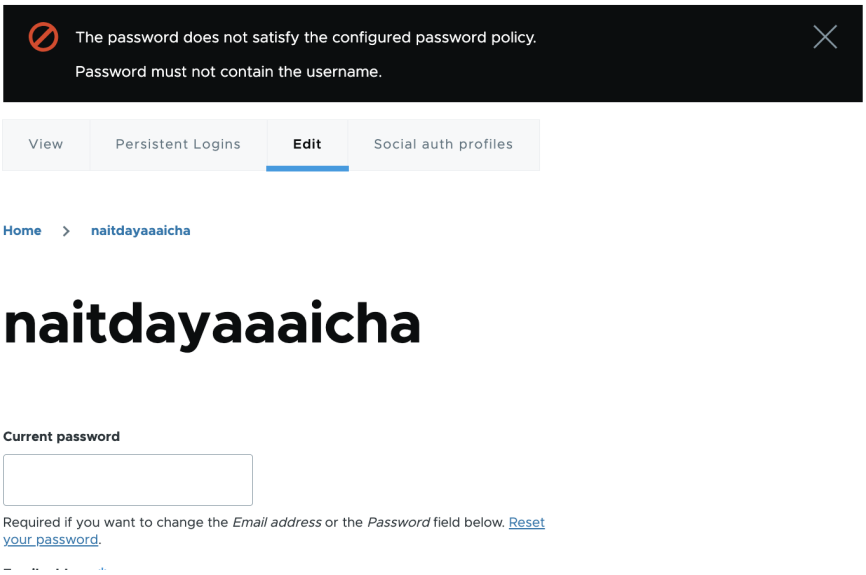


Figure 12: Violation raised

Q2. How to quickly generate and scaffold a new entity codebase ?

Using the **Devel Generate** module or **Drupal Console**:

```
drupal generate:entity:content
```

This generates the full entity boilerplate including annotation, base fields, access handler, forms, list builder, routing, and permissions interactively.

Q3. How do you get a field definition using code ?

To retrieve a field definition using code, we call the service **entity_field.manager**, that manages the discovery of entity fields. It is registered in the **core.services.yml** file, and defined in the **EntityFieldManager.php** class. The method used to retrieve the field definition is **getFieldDefinitions($entity_type, $bundle)**

Example :

```
$fields = \Drupal::service('entity_field.manager')->
    getFieldDefinitions('node', 'article');

$definition = $fields['field_name'];
```

Q4. What is AccessResult and how does it work

AccessResult standardizes access check outcomes by returning one of three states: **allowed** : access is granted, **forbidden** : access is denied, or **neutral** : access nndefined, leaving the decision to other access checks in the chain.

Q5. Is it possible to create multiple fields formatter for a field type ?

Yes, it is possible. To do so, each formatter is implemented in a separate class, with a unique ID defined in its `#[FieldFormatter]` attribute. One of them is set as the `default_formatter` for the field type. Once the module is enabled, we can choose which formatter to use in the *Manage Display* page.

Q6. Using drush, how to retrieve a module configuration (settings) ?

To retrieve a module's configuration using drush, we use the command :

```
drush config:get module.settings
```

2 Working with hooks

2.1. Introduction

In this section, we complete our exploration of Drupal's hook system by covering additional hooks related to entities, theming, and configuration.

2.2. Questions

Q1. Adding a new field base to an exisiting entity type using hook_entity_base_field_info

:

To test this hook, we will add a new base field to the contact form of the custom module **content_entity_example**.

The necessary steps to accomplish this functionality :

1. **Implementing the hook hook_entity_base_field_info**

   This hook in general, allows defining new base fields for existing entity types. In **src/Hook** we define the class **ContentEntityExampleHook** where we implement the hook.

   The **content_entity_example_entity_base_field_info** method define an additional field.

```
class ContentEntityExampleHook {

  #[Hook('entity_base_field_info')]
  public function content_entity_example_entity_base_field_info(
     EntityTypeInterface $entity_type) : array {

    $fields = [];
    if($entity_type->id() === 'content_entity_example_contact') {
      $fields['additional_field'] = BaseFieldDefinition::create('
         string')
        ->setLabel('Additional Field')
```

```
        ->setDescription('This is an additional field added via hook_entity_base_field_info().')
        ->setRequired(FALSE)
        ->setDisplayOptions('view', [
          'label' => 'above',
          'type' => 'string',
          'weight' => 0,
        ])
        ->setDisplayOptions('form', [
          'type' => 'string_textfield',
          'weight' => 0,
        ])
        ->setDisplayConfigurable('form', TRUE)
        ->setDisplayConfigurable('view', TRUE);
    }
    return $fields;
  }
```

2. **Persisting the new base field to the database** :

   The new base field isn't persisted to the database automatically, to do so, we should implement the hook **hook_update_N**, that performs updates using a versioning system (the number N is the update number). This hook should be implemented in a **.install** file, which is executed when the database update process runs (e.g., via **drush updb**).

```
/**
 * Add 'additional_field' base field to Contact entity.
 */
function content_entity_example_update_10101() {
  // Get the Entity Definition Update Manager service.
  $update_manager = \Drupal::service('entity.definition_update_manager');

  $fields = \Drupal::service('entity_field.manager')->getBaseFieldDefinitions('content_entity_example_contact');

  if(!isset($fields['additional_field'])) {
    throw new RuntimeException('The field "additional_field" is required.');
  }

  $update_manager->installFieldStorageDefinition(
    'additional_field',
    'content_entity_example_contact',
    'content_entity_example_contact',
    $fields['additional_field']
```

```
  );

  return t('Added base field "additional_field" to Contact entity.');
}
```

- The service **entity.definition_update_manager** manages entity definition updates.
  Class definition :
  /core/lib/Drupal/Core/Entity/EntityDefinitionUpdateManager.php
- The service **EntityFieldManager**, manages the discovery of entity fields.
  Class definition :
  /core/lib/Drupal/Core/Entity/EntityFieldManager.php

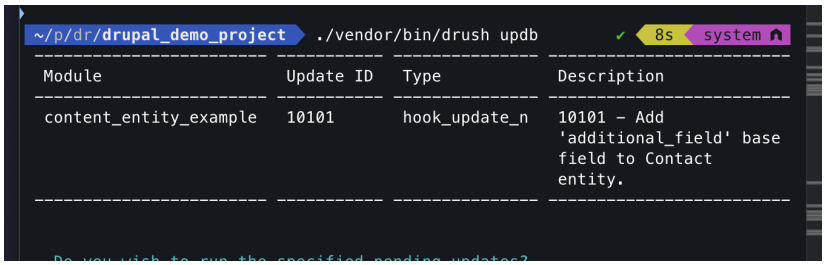


Figure 13: updating the database schema

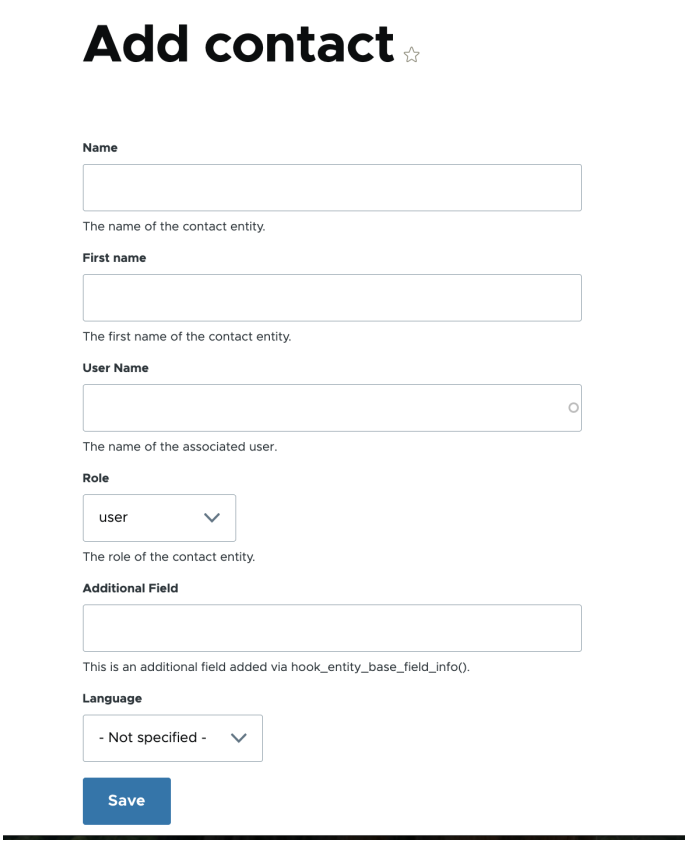


Figure 14: The contact form with an additional field

Q2. What is the role of hook_install ?

The hook **hook_install**, runs in initial setup of the project (first installation of the module), and is defined only in a procedural way in the **.install** file.

It is used to set default configuration values, create initial content, or set up database entries required for the module to function correctly upon installation.

Q3. How can you override a Theme Hook provided by another module ?

Basic Override

A theme hook can be overridden by copying the template implementation of the module into the **themes/templates** directory keeping the same filename.

For example, we want to override the display of search results, to do so, we need to copy the default template file **search-result.html.twig** into the **themes/templates** directory.

Specific Override

For targeted overrides (specific context), we use the naming convention **template-name–specification.html.twig**.

For example, we want to override the display of search results only for **Article content type**, we copy the template file and name it something like **search-result–node–article.html.twig**

> If there are many templates for this module defined in the **themes/templates** directory, the most specific template is the one that will get picked up by drupal (e.g. **search-result–node–article.html.twig** will get picked up over **search-result–node.html.twig**)

Q4. hook_ENTITY_TYPE_presave

To prefix all article node titles with **HEY-** before saving:

```
function mymodule_node_presave(EntityInterface $entity) {
  if ($entity->bundle() === 'article') {
    $entity->setTitle('HEY-' . $entity->getTitle());
  }
}
```

Q5. What is the role of $entity→original

$entity→original holds the unchanged version of the entity as it exists in the database before the save operation. It is useful for comparing old and new values, for example detecting whether a specific field has changed:

```
if ($entity->getTitle() !== $entity->original->getTitle()) {
  // Title has changed
}
```

Q6. hook_theme_suggestions_alter

To add a theme suggestion for the **user** entity based on the view mode:

```
function mymodule_theme_suggestions_alter(array &$suggestions, array $
    variables, $hook) {
  if ($hook === 'user') {
    $view_mode = $variables['elements']['#view_mode'];
    $suggestions[] = 'user__' . $view_mode;
  }
}
```

This allows creating a dedicated template **user–view-mode.html.twig** for each view mode, for example **user–full.html.twig** or **user–compact.html.twig**.

3 Working with configuration & features

3.1. Introduction

In this section, we explore the two approaches for managing Drupal configuration: the **Config Split** module, which allows environment-specific configuration to be separated, and the **Features** module, which packages configuration into reusable modules. Both methods were implemented and tested in separate development and production instances in order to analyze their behavior and identify the situations in which each approach is most suitable.

3.2. Configuration Export and Import

By default, Drupal stores configuration in the database. However, for deployment and version control purposes, it is preferable to store configuration as files within the project.

Drupal provides commands that allow configuration to be exported from the database and written to a specific directory in the project. This directory is defined in the **settings.php** file.

```
$settings['config_sync_directory'] = '../config/sync';
```

Once configuration files are exported and available, they can be imported into another Drupal instance to reproduce the same site structure and settings.

3.3. Managing Environment Differences with Config Split

When working with multiple environments, some configuration elements should remain specific to development. For example, debugging tools should not be enabled in production. To handle this situation, the **Config Split** module was used to separate configuration depending on the environment.

To test this workflow, two Drupal instances were created: one for the **development environment** and one for the **production environment**.

1. **Development Environment**

   In the development instance, several configuration elements were added in order to generate configuration changes:

   - Installation of the **Devel** module
   - Creation of a taxonomy named **Categories**
   - Creation of a content type **Page**
   - Creation of a role **Page Editor** with permissions to manage the **Page** content type

   Drupal stores configuration in the database. To transfer it between environments, the configuration is exported to files using the synchronization directory defined in **settings.php**.

```
$settings['config_sync_directory'] = '../config/sync';
```

   Before exporting the configuration, the split configuration directories were created according to the structure recommended in the documentation. The exported configuration files are stored in these directories.

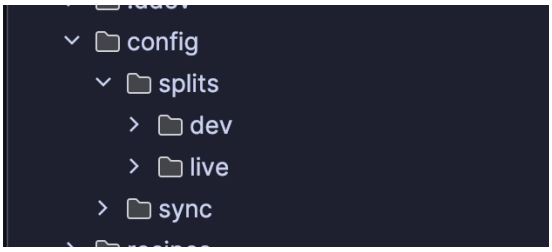


Figure 15: Configuration directory structure

   Two configuration splits were then defined through the Drupal interface: one for the development environment and one for production. The development split excludes the **Devel** module from the shared configuration.

   The active split is controlled in the **settings.php** file:

```
$config['config_split.config_split.dev']['status'] = TRUE;
$config['config_split.config_split.live']['status'] = FALSE;
```

   After configuring the split, the configuration was exported from the development instance.

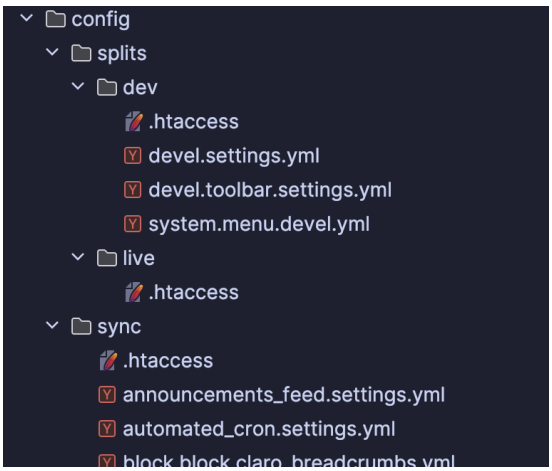


Figure 16: Exported configuration files

2. **Production Environment**

   The exported configuration directory is then pulled to the production instance and imported using Drupal's configuration import process.

   During the import, two issues were encountered:

   (a) **Site UUID mismatch:** Each Drupal installation has a unique identifier. Since the configuration came from the development environment, the production site needed the same UUID to accept the configuration.

```
# Get DEV uuid
drush config-get system.site uuid

# Set it on PROD
drush config-set system.site uuid "UUID"
```

   (b) **Shortcut entities conflict:** Some existing shortcut entities in the production site conflicted with the imported configuration. These entities needed to be deleted before the import could succeed.

After resolving these issues, the configuration import was completed successfully.

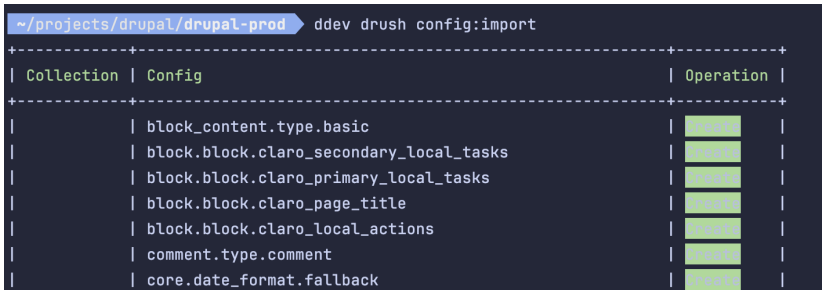


Figure 17: Configuration successfully imported

3.4. Features Module

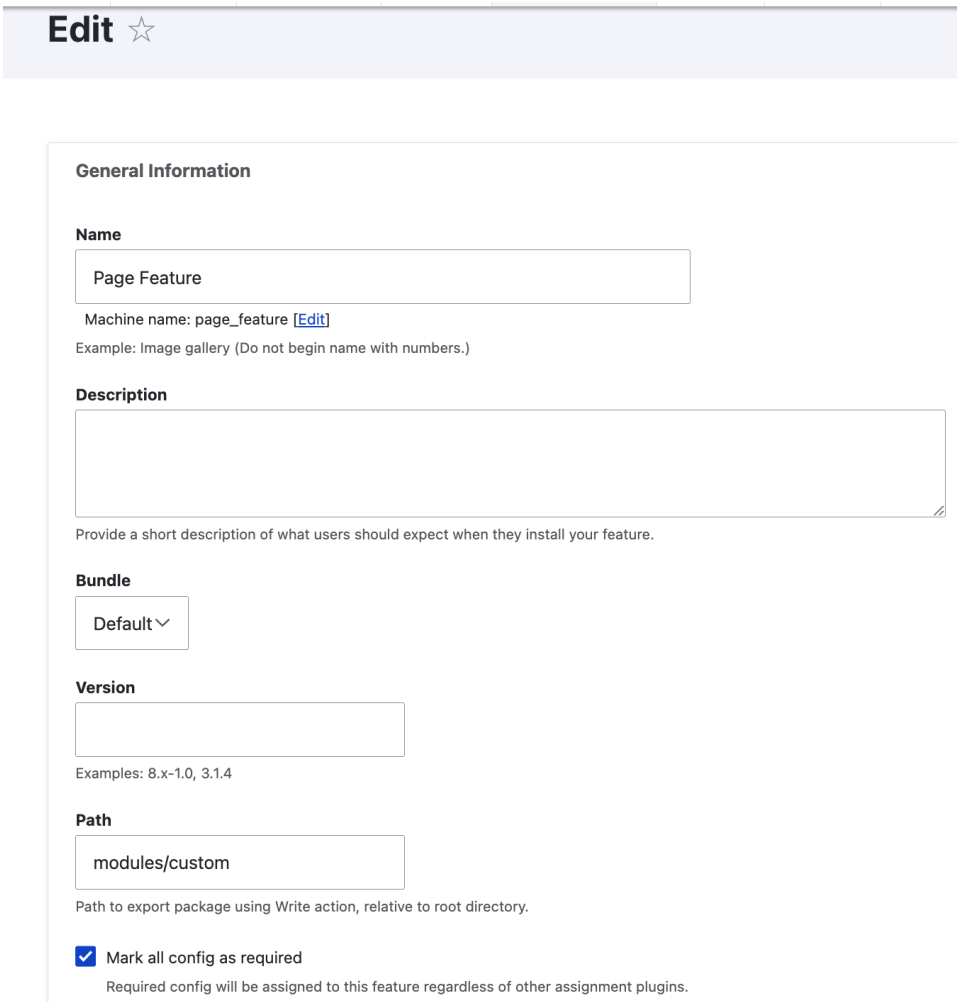
The **Features** module allows bundling selected configuration into a reusable Drupal module. Unlike Config Split, which manages environment-specific differences for the entire configuration, Features lets us package only the relevant items into a module that can be deployed to another site.

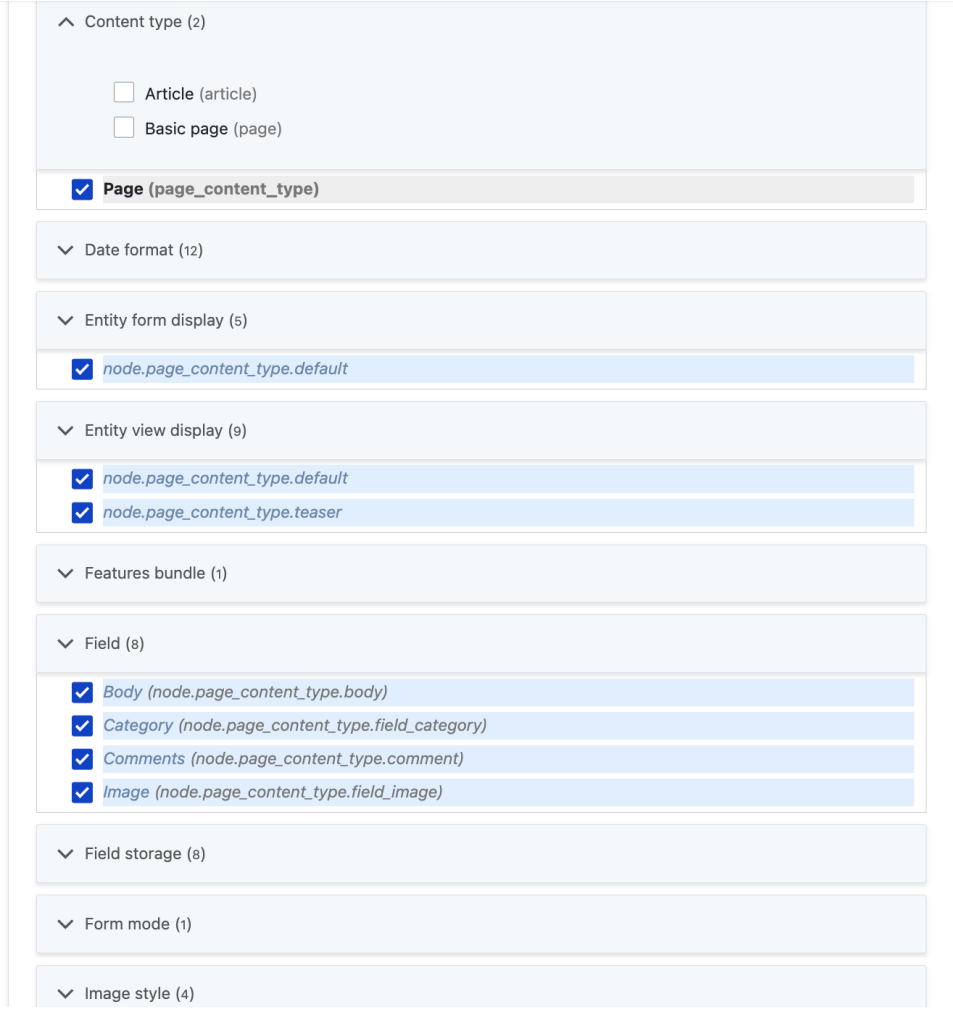
1. **Development Environment**

   In the development instance, the Features module and its UI were installed:

```
ddev composer require drupal/features
ddev drush en features features_ui -y
ddev drush cr
```

   A feature bundle **page_feature** was created via the UI, containing the same configuration made in the last part.





Clicking **Write** generated the module on disk:

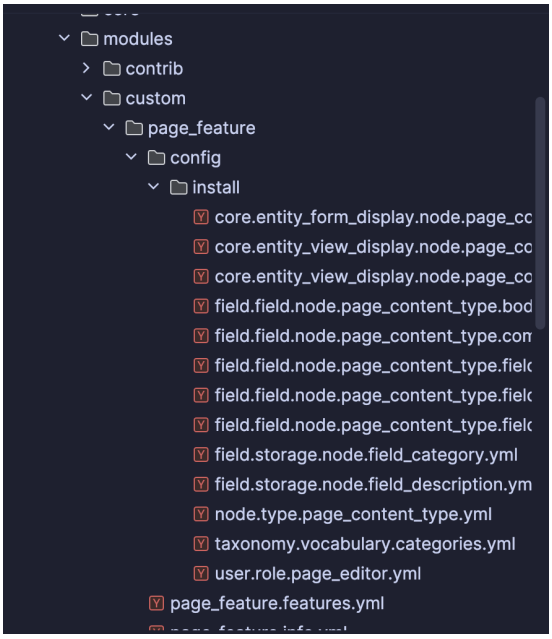


Figure 18: Feature module generated in development

2. **Production Environment**

In production, only the Features module was installed, because production only needs to apply the packaged configuration, not create or edit feature bundles.

```
composer require 'drupal/features:^3.16'
ddev drush en features
```

The feature module was copied from development and enabled using Drush. This automatically applied the configuration stored in the module:

```
ddev drush en page_feature -y
ddev drush cr
```

3. **Conflict Resolution**

To test conflicts:

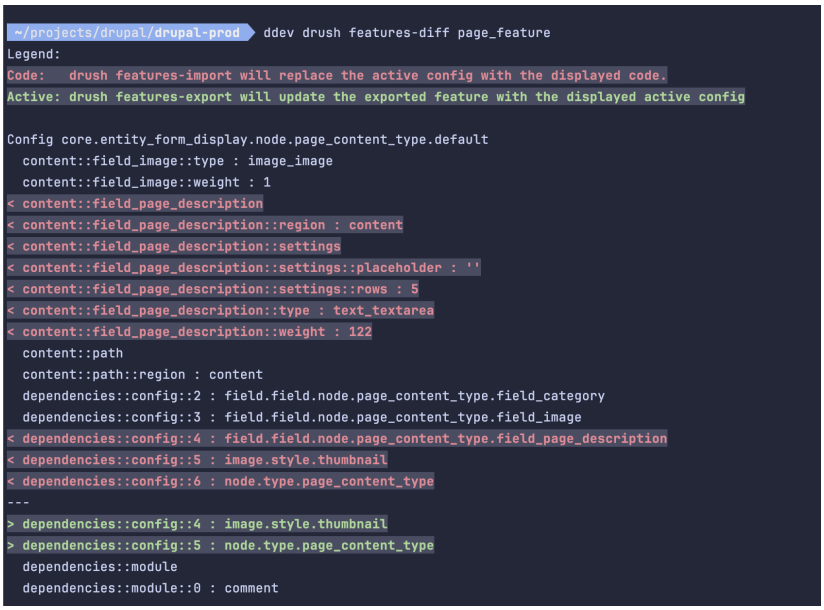
(a) In production, the content type name was manually changed.

(b) In development, a new **Description** field was added, and the feature was re-exported:

```
ddev drush features-export page_feature
```

(c) The updated module was copied to production, and the diff was checked:

```
ddev drush features-diff page_feature
```



(d) The updated feature was imported on production:

```
ddev drush features-import page_feature
ddev drush cr
```

After import:

- The content type name was reverted to match development.
- The new field was added to production, ensuring configurations matched.

> Since manual configuration in production is overwritten by the feature module, all changes must originate in development and be deployed via `features-export` and `features-import`.

3.5. Conclusion

Both modules serve distinct but complementary roles in Drupal configuration management. **Config Split** is best suited for managing environment-specific differences, ensuring a clean separation between development and production. **Features**, on the other hand, is best at packaging configuration into reusable, portable modules for cross-project use. Together, they provide a good workflow for maintaining, sharing, and deploying Drupal configuration across different environments and installations.

4 Working with Cache

4.1. Introduction

In this section, we explore Drupal's caching system by building a JSON endpoint and testing two caching strategies: **max-age** and **cache tags**.

4.2. Implementation

Q1. Creating the Route

We created a custom module **article_api** exposing a route **/api/articles** that returns a JSON response containing 3 article nodes.

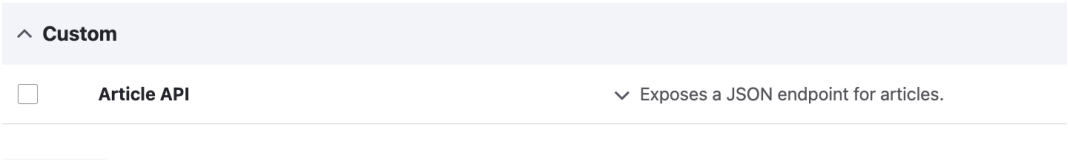


Figure 19: Article Api Module

```
article_api.articles:
  path: '/api/articles'
  defaults:
    _controller: '\Drupal\article_api\Controller\ArticleApiController::
       getArticles'
    _title: 'Articles API'
  requirements:
    _access: 'TRUE'
```

The controller uses **entityQuery** to fetch 3 article nodes with hardcoded IDs, and returns a JSON response in the format **[{nid: integer, title: string}]**.

```
$result = \Drupal::entityQuery('node')
  ->condition('type', 'article')
  ->condition('status', 1)
  ->condition('nid', [7, 4, 34], 'IN')
  ->accessCheck(TRUE)
  ->execute();
```

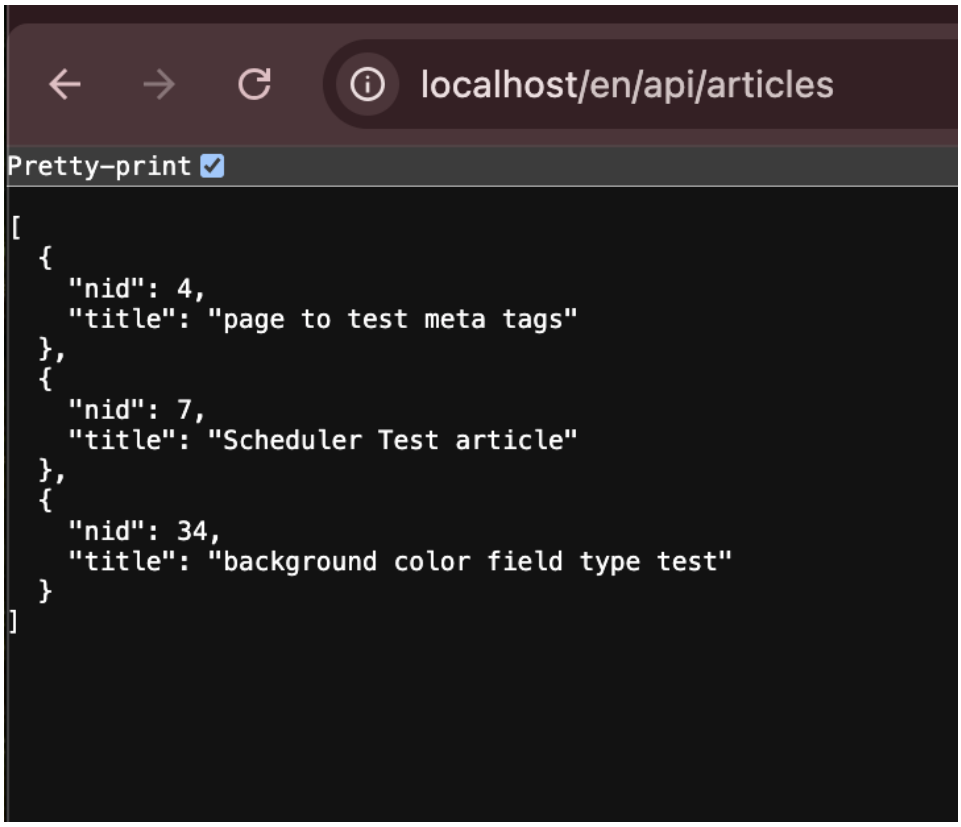


Figure 20: Json Response

Q2. Testing max-age

We first configured the response with a **max-age** of 3600 seconds:

```
$response = new JsonResponse($articles);
$response->setMaxAge(3600);
return $response;
```

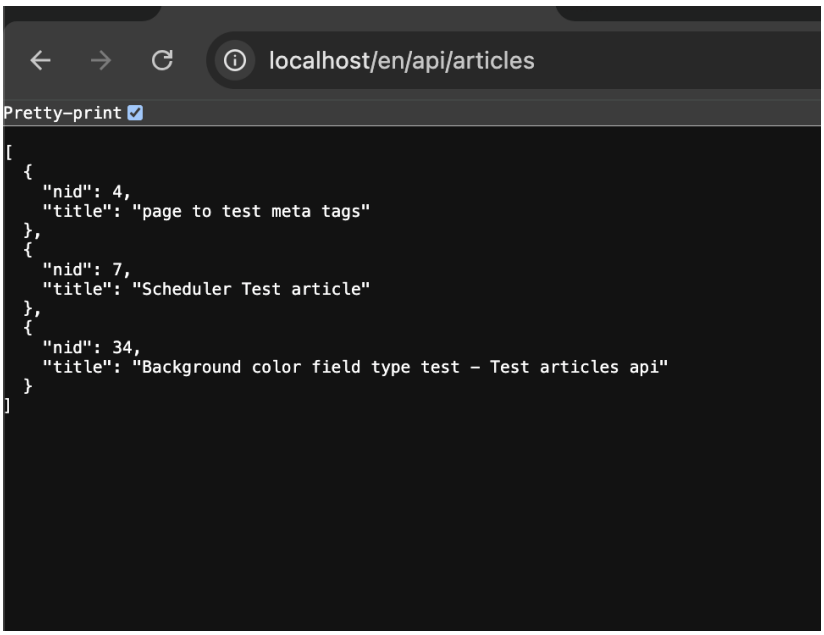
After editing one of the node titles, hitting the endpoint again returned the **old title** — the cache was not invalidated until the 3600 seconds expired. This confirms that **max-age** is time-based and unaware of content changes.

Q3. Testing Cache Tags

To invalidate the cache on content change, we use **cache tags**. **CacheableMetadata** collects the tags from each node, and **CacheableJsonResponse** attaches them to the response:

```php
public function getArticles() : CacheableJsonResponse {
    $result = \Drupal::entityQuery('node')
                        ->condition('type', 'article')
                        ->condition('status', 1)
                        ->condition('nid', [7, 4, 34], 'IN')
                        ->accessCheck(TRUE)
                        ->execute();

    $nodes = \Drupal::entityTypeManager()
                        ->getStorage('node')
                        ->loadMultiple($result);

    $articles = [];
    $cache_metadata = new CacheableMetadata();

    foreach($nodes as $node) {
      $articles[] = [
        'nid' => (int) $node->id(),
        'title' => $node->getTitle(),
      ];

      $cache_metadata->addCacheTags($node->getCacheTags());
    }

    $response = new CacheableJsonResponse($articles);
    $response->addCacheableDependency($cache_metadata);

    return $response;

  }
```

After editing a node title, the endpoint returned the **updated title instantly**. Drupal automatically invalidated the cache for any response tagged with **node:X** when that node was saved.



Q4. Inspecting Cache Tags via Response Headers

To inspect cache tags, we enabled debug cacheability headers in **development.services.yml**:

```
parameters:
  http.response.debug_cacheability_headers: true
```

And verified it was loaded in **settings.php**:

```
$settings['container_yamls'][] = DRUPAL_ROOT . '/sites/development.
services.yml';
```

Then inspected the response headers:

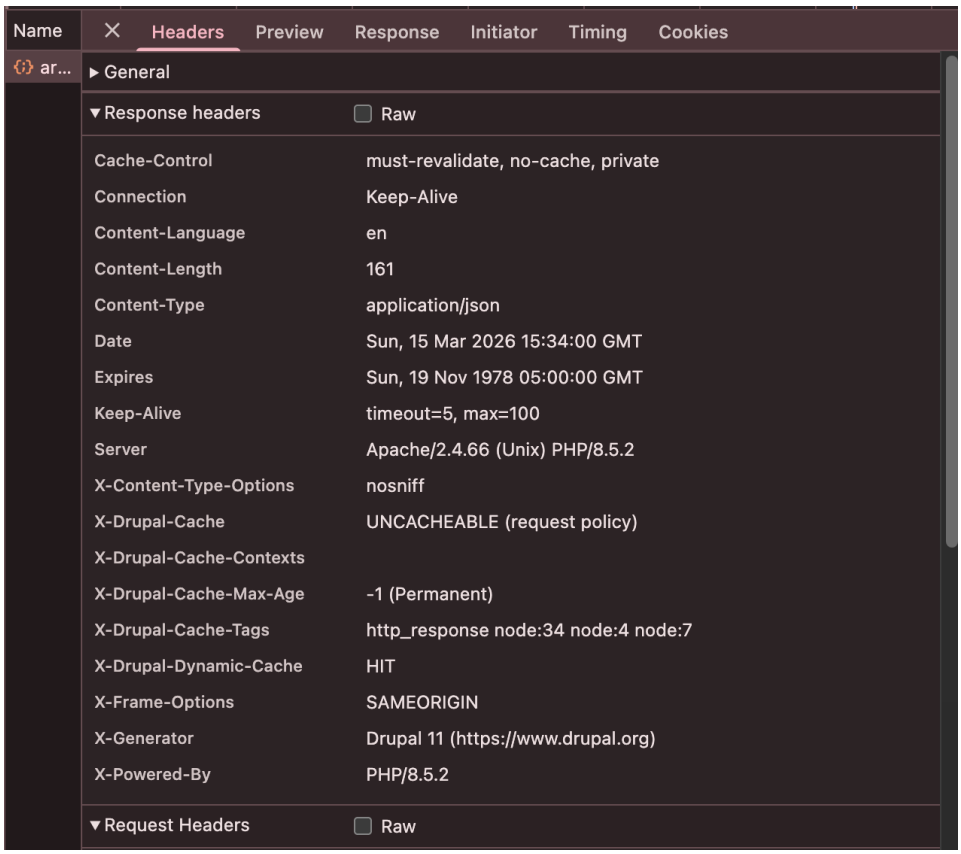


Figure 21: Caption

4.3. Conclusion

The **max-age** strategy caches responses for a fixed duration regardless of content changes, while **cache tags** provide precise invalidation tied directly to entity updates. For API endpoints returning dynamic Drupal content, cache tags are the recommended approach as they ensure data freshness without sacrificing performance.

5 Working with Data Migration

5.1. Questions

Q1. Role of `migration_lookup`

The **migration_lookup** process plugin maps a source ID to the ID of a previously migrated entity. For example, when migrating nodes that reference taxonomy terms, instead of hardcoding the term ID, **migration_lookup** looks up the destination ID of the term that was already migrated, ensuring relational integrity between entities.

Other commonly used process plugins are documented in Process Plugins.

Q2. Importing from a MySQL Database

Instead of using the **CSV** source plugin, we define an external database connection in **settings.php** and implement a custom source plugin extending **SqlBase**:

```php
# settings.php
$databases['migrate']['default'] = [
  'driver'   => 'mysql',
  'host'     => 'localhost',
  'database' => 'source_db',
  'username' => 'root',
  'password' => 'password',
];
```

```yaml
source:
  plugin: my_custom_sql_source  # custom plugin extending SqlBase
  key: migrate
```

Q3. Rolling Back a Migration

To rollback a migration and delete all imported entities, we execute this command provided by the Migrate Tools contributed module:

```
drush migrate:rollback migration_id
```

This removes all destination entities created by the migration, restoring the site to its pre-migration state.

Q4. Processing Field Data Before Import

To process a source field before importing it, we use process plugins in the migration YAML. To trim a string to 10 characters, we use the **callback** plugin with PHP's **substr** function:

```yaml
process:
  title:
    plugin: callback
    callable: substr
    source: title
    unpack_source: true
    arguments:
      - 0
      - 10
```